



Strix: An End-to-End Streaming Architecture with Two-Level Ciphertext Batching for Fully Homomorphic Encryption with Programmable Bootstrapping

Adiweni Putra
KAIST
Daejeon, South Korea
adiweni.research@kaist.ac.kr

Prasetyo
KAIST
Daejeon, South Korea
pras@kaist.ac.kr

Yi Chen
KAIST
Daejeon, South Korea
chenyi@kaist.ac.kr

John Kim
KAIST
Daejeon, South Korea
jjk12@kaist.ac.kr

Joo-Young Kim
KAIST
Daejeon, South Korea
jooyoung1203@kaist.ac.kr

ABSTRACT

Homomorphic encryption (HE) is a type of cryptography that allows computations to be performed on encrypted data. The technique relies on learning with errors problem, where data is hidden under noise for security. To avoid excessive noise, bootstrapping is used to reset the noise level in the ciphertext, but it requires a large key and is computationally expensive. The fully homomorphic encryption over the torus (TFHE) scheme offers a faster and programmable bootstrapping (PBS) algorithm, which is crucial for many privacy-focused applications. Nonetheless, the current TFHE scheme does not support ciphertext packing, resulting in low-throughput performance. To the best of our knowledge, this is the first work that thoroughly analyzes TFHE bootstrapping, identifies the TFHE acceleration bottleneck in GPUs, and proposes a hardware TFHE accelerator to solve the bottleneck.

We begin by identifying the TFHE acceleration bottleneck in GPUs due to the blind rotation fragmentation problem. This can be improved by increasing the batch size in PBS. We propose a two-level batching approach to enhance the batch size in PBS. To implement this solution efficiently, we introduce Strix, utilizing a streaming and fully pipelined architecture with specialized units to accelerate ciphertext processing in TFHE. Specifically, we propose a novel microarchitecture for decomposition in TFHE, suitable for processing streaming data at high throughput. We also employ a fully-pipelined FFT microarchitecture to address the memory access bottleneck and improve its performance through a folding scheme, achieving $2\times$ throughput improvement and $1.7\times$ area reduction. Strix achieves over $1,067\times$ and $37\times$ higher throughput in running TFHE with PBS than the state-of-the-art implementation on CPU and GPU, respectively, surpassing the state-of-the-art TFHE accelerator, MATCHA, by $7.4\times$.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0329-4/23/10...\$15.00
<https://doi.org/10.1145/3613424.3614264>

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**; • **Security and privacy** → **Cryptography**.

KEYWORDS

fully homomorphic encryption, accelerator, programmable bootstrapping, ciphertext batching

ACM Reference Format:

Adiweni Putra, Prasetyo, Yi Chen, John Kim, and Joo-Young Kim. 2023. Strix: An End-to-End Streaming Architecture with Two-Level Ciphertext Batching for Fully Homomorphic Encryption with Programmable Bootstrapping. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23), October 28–November 01, 2023, Toronto, ON, Canada*. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3614264>

1 INTRODUCTION

Homomorphic encryption (HE) allows data to be processed while data remains encrypted and provides significant improvement in privacy and security [45]. One fundamental component of most HE schemes is that noise is introduced to maintain data security and limit the amount of computation that is possible. However, the introduction of bootstrapping enabled fully homomorphic encryption (FHE) where noise in the ciphertext is refreshed and enables an unlimited amount of computation on the data [24]. The benefits of FHE are significant, not only from a security and privacy perspective but also in enabling the secure offloading of computation to the cloud without revealing data. Unfortunately, one of the most significant challenges of FHE is the high computation cost. For instance, a baseline inference can take 100 ms on CPU, but an FHE implementation can take up to 3 hours, resulting in more than $100,000\times$ slowdown [35].

Recently, there has been significant effort in accelerating FHE, utilizing not only GPUs [15, 30, 40, 48] and FPGAs [1, 22, 44, 50], but also dedicated hardware accelerators [32, 33, 46, 47]. Most prior work on acceleration has focused on the popular FHE scheme, CKKS [6], which supports real numbers representation and is suitable for machine learning workloads. However, while CKKS is powerful, it is still costly both in terms of computation and cost. For example, a recent accelerator for CKKS [32] delivers significant performance improvement, but its estimated cost is approximately

418 mm² (nearly equivalent to a high-end GPU) with 512 MB of on-chip memory.

In this work, we investigate a different type of FHE based on Fully Homomorphic Encryption over the Torus (TFHE) [10], which offers lower computational and memory requirements compared to other FHE schemes. Similar to other FHE schemes, the most time-consuming operation in TFHE is bootstrapping, which is necessary to refresh noise during homomorphic computations. However, a crucial advantage of TFHE bootstrapping is its programmability, allowing the evaluation of any univariate function while performing bootstrapping simultaneously through programmable bootstrapping (PBS). The computational requirements, as well as the input data structure, fundamentally differ from other FHE schemes like CKKS. Consequently, prior work on FHE accelerators [1, 32, 33, 46, 47, 50] are not directly applicable to TFHE.

The main challenge in TFHE is achieving high-throughput programmable bootstrapping (PBS) since it operates on one message at a time. Our analysis reveals that the primary bottleneck is blind rotation, which requires multiple iterations of vector-matrix multiplication of polynomials to be performed sequentially. To enhance PBS performance, we can address either its latency or throughput. To improve PBS latency, a single blind rotation can be accelerated when processing one ciphertext (i.e., accelerate individual iteration of blind rotation). When there are multiple ciphertexts, blind rotation needs to be performed multiple times for each ciphertext [28, 51]. In comparison, PBS throughput can be improved by enabling a single blind rotation to process multiple ciphertexts simultaneously [39, 40]. While reducing the latency of PBS improves performance, the workload becomes memory-bound because of a shorter time gap to fetch the next bootstrapping key for the next blind rotation iteration. In comparison, addressing the throughput of PBS becomes compute bound as more temporal reuse of bootstrapping key, resulting in a larger time gap to fetch the next bootstrapping key. However, in this work, we analyze PBS on GPUs and identify how attempting to improve throughput can result in *blind rotation fragmentation* where multiple blind rotations need to be performed sequentially and significantly deteriorate performance.

To address blind rotation fragmentation, increasing batch size of a single blind rotation is essential, which is often constrained by the number of cores. We introduce the concepts of device-level and core-level batching to significantly expand the batch size for a single blind rotation. Device-level ciphertext batching substantially enhances spatial parallelism by enabling each core to work on different ciphertexts while utilizing the same set of bootstrapping keys, thereby increasing spatial reuse. Conversely, core-level batching improves temporal parallelism by allowing each core to process a continuous stream of ciphertexts using the same set of bootstrapping keys. However, efficiently implementing core-level batching necessitates specialized hardware units designed to exploit available parallelism and amortize the cost of processing ciphertexts sequentially in each blind rotation iteration.

We introduce Strix, a hardware accelerator for high-throughput TFHE. It uses two-level batching to increase the batch size of a single blind rotation, addressing the blind-rotate fragmentation problem. Strix features five functional units for efficient processing in one blind rotation iteration. After analyzing the PBS algorithm, we identified four levels of parallelism in PBS, guiding the design of

the Strix accelerator architecture for optimal pipelining, achieving nearly 100% utilization and maximum throughput.

Among these functional units, the decomposer unit and (I)FFT unit play critical roles and present significant design challenges. The decomposer unit's task is to decompose one polynomial into smaller polynomials. We present a novel microarchitecture for the decomposer unit that effectively consumes a stream of polynomials and produces a decomposed stream of polynomials with constant throughput, using efficient masking and buffering techniques. Another important functional unit is the I/FFT unit. Unlike the I/FFT units implemented in CKKS accelerators (I/NTT in their case), which employ 2D NTT requiring matrix transposition between different steps, we adopt a parallel pipelined FFT unit that processes a constant stream of polynomials without the need for matrix transposition. This design aligns well with our streaming architecture. We further optimize the FFT unit with a folding scheme technique, allowing the transformation of N-point polynomials using an N/2-point FFT unit, effectively reducing the latency and hardware cost by 50%.

We integrate all functional units into a single Homomorphic Streaming Core (HSC). With up to 8 HSCs on one chip and optimized data flow, we achieve a 1,067× and 37× throughput increase, and a 38× and 17× speedup for machine learning workloads compared to top CPU and GPU implementations.

In this paper, we make the following key contributions:

- We perform an in-depth analysis of TFHE programmable bootstrapping (PBS) on a GPU and identify a critical performance issue stemming from the blind rotation fragmentation problem. This issue arises because the blind rotation process must be executed multiple times, leading to significant performance degradation.
- We propose Strix, a hardware accelerator designed to mitigate the blind rotation fragmentation problem through the concepts of device-level and core-level batching to substantially enhance the batch size for a single-blind rotation.
- To efficiently implement core-level batching, specialized hardware units are required to exploit any available parallelism and amortize the cost of sequentially processing ciphertexts in each blind rotation iteration. We perform a comprehensive analysis of PBS parallelism, identifying four distinct levels of parallelism that serve as the foundation for designing the Strix accelerator architecture.
- We introduce a novel microarchitecture for Strix, featuring five interconnected functional units that achieve nearly 100% utilization. We present an innovative decomposer unit that efficiently processes polynomial streams and a fully pipelined FFT unit that eliminates matrix transposition and incorporates a folding scheme for enhanced blind rotation computation.

2 BACKGROUND

2.1 Homomorphic Encryption

Homomorphic Encryption (HE) allows computations on encrypted data without decryption [45]. Modern HE schemes rely on lattice problems, which are known to be secure against quantum computers [3, 31, 43]. Learning With Errors (LWE) is commonly used

Table 1: Typical parameter value for TFHE scheme

Parameter	Description	Typical value
n	LWE mask	$2^8 - 2^{12}$
N	Polynomial degree	$2^{10} - 2^{14}$
k	GLWE mask	1–4
l_b	decomposition level	2–4

to implement HE systems [24] as LWE ensures data security by concealing it under noise. However, as computations are performed on encrypted data, the noise accumulates, and if the noise exceeds a certain threshold, the original data cannot be decrypted. Fully Homomorphic Encryption (FHE) overcomes this limitation with bootstrapping mechanism that can refresh the noise in the ciphertext back to its initial level [24]. Unfortunately, bootstrapping is computationally expensive and requires minutes to hours of execution for a single ciphertext [23, 25]. As a result, different HE schemes have been proposed to improve the efficiency of bootstrapping [5–7, 17, 19, 26].

HE schemes are typically classified into two categories: BGV-like and GSW-like schemes [18, 37, 41]. BGV-like schemes, such as BGV [19], BFV [5], and CKKS [6], encrypt messages at the vector level, with each homomorphic operation acting as a vector operation. While bootstrapping in BGV-like schemes is relatively slow, they provide high-throughput bootstrapping. On the other hand, GSW-like schemes, including GSW [26], FHEW [17], and TFHE [7], encrypt messages at the word or bit level. This approach offers greater flexibility, supporting a wide range of operations such as arithmetic, logical, and relational operations. This versatility, useful for various HE applications, is due to functional or programmable bootstrapping (PBS) that allows for homomorphic look-up tables.

2.2 TFHE Overview

Fast fully homomorphic encryption scheme over the torus (TFHE) was proposed to enable bootstrapping to be performed in under 0.1 seconds using a single-core CPU [7]. Unlike prior encryption schemes [5, 6, 19] that require separate (dedicated) bootstrapping steps to reset noise, TFHE supports programmable bootstrapping which allows bootstrapping to be performed during the evaluation of arbitrary univariate functions [12]. TFHE was initially limited to performing boolean algebra operations but has been extended to include operations for integer and real numbers [8–10, 12–14]. Homomorphic addition and programmable bootstrapping (PBS) are the two fundamental homomorphic operations supported by TFHE, which can be used for constructing any function, including logical, and relational operations. In general, PBS enables the construction of any function of one variable through the use of PBS.

2.3 Data Structure in TFHE

Entities involved in PBS and keyswitching operations in TFHE can be categorized into four groups: LWE ciphertext, GLWE test-vector, bootstrapping key (bsk), and keyswitching key (ksk). LWE ciphertext is represented as a vector containing $(n + 1)$ scalar elements $[a_1, \dots, a_n, b]$, where each scalar element is a 32-bit or 64-bit integer. This ciphertext type is primarily used to encrypt messages in TFHE. On the other hand, GLWE is a ciphertext type that stores univariate

Algorithm 1 Programmable Bootstrapping

Require: n, k : mask length in LWE/GLWE ciphertext
Require: l_b : decomposition level of bootstrapping
Require: N : number of coefficients in polynomial
Require: $c[n + 1]$: LWE ciphertext, where each element is scalar
Require: $tv[k + 1]$: test vector, each element is N degree polynomial
Require: $bsk[n][(k + 1)l_b][k + 1]$: bootstrapping key, each element is N degree polynomial
Ensure: $o[kN + 1]$: LWE ciphertext, where each element is scalar

```

1: Local variables:  $etv[(k + 1)l_b]$ ,  $otv[k + 1]$  (initialized to zero)
2: for  $i = 0$  to  $(n + 1)$  do
3:    $c[i] = \text{ModSwitch}(c[i])$  ▷ // Modulus Switching
4:    $tv = \text{Rotate}(\text{'left'}, tv, c[n])$ 
5:   for  $i = 0$  to  $n$  do ▷ // Blind Rotation
6:      $tv = tv - \text{Rotate}(\text{'Right'}, tv, c[i])$  ▷ // Rotate and subtract
7:      $etv = \text{Decompose}(tv, l_b)$  ▷ // Decomposition
8:     for  $j = 0$  to  $(k + 1)$  do
9:       for  $k = 0$  to  $(k + 1)l_b$  do
10:         $otv[j] = etv[j] + \text{IFFT}(\text{FFT}(etv[k]) * \text{FFT}(bsk[i][k][j]))$  ▷
11:      // External product
12:    $tv = etv$ 
13:  $o = \text{SampleExtract}(tv)$  ▷ // External Sample Extract

```

functions for PBS. It is represented as a vector of $(k + 1) N$ degree polynomials $[A_1(X), \dots, A_k(X), B(x)]$, where each polynomial is represented as a vector of N integers, either 32-bit or 64-bit.

The bootstrapping key and the keyswitching key act as "parameters" that interact with the ciphertext and test-vector during PBS or keyswitching operations. The bootstrapping key consists of a vector of n General-GSW (GGSW) ciphertexts, where each GGSW ciphertext is a $(k + 1) \cdot l_b \times (k + 1)$ matrix of N degree polynomials. Conversely, the keyswitching key comprises a vector of $k \cdot N \cdot l_k$ LWE ciphertexts. Table 1 displays the typical parameter values for the TFHE scheme.

In summary, LWE and GLWE represent the encrypted message and the univariate function, respectively, while the bootstrapping key and keyswitching key serve as the parameters to operate with the ciphertext and test-vector during PBS or keyswitching operations. These entities are essential in the TFHE scheme's homomorphic encryption operations, and their efficient implementation is crucial for performance. For more details, refer to the original cryptographic algorithm paper [9].

2.4 PBS and Keyswitching

Programmable bootstrapping (PBS). Algorithm 1 outlines the PBS procedure in TFHE. For enhanced readability, we present the original algorithm [10, 29] using matrix and vector operations. PBS consists of three building blocks: modulus switching, blind rotation, and sample extract. During modulus switching, each element in LWE transitions from the original modulus q to the new modulus $2N$. This operation is straightforward and cost-effective since N is always a power of two. Blind rotation, the core operation of PBS, accounts for 96% of the PBS execution time, highlighting its computational intensity. It involves non-parallelizable iterations, each comprising three steps: polynomial rotation and subtraction, decomposition, and external product. Finally, sample extract creates

Algorithm 2 Keyswitching

Require: n, k : input mask length of LWE/GLWE ciphertext
Require: l_k : decomposition level of keyswitching
Require: $c[n+1]$: LWE ciphertext, where each element is scalar
Require: $ksk[nl_k][m+1]$: keyswitching key, each element is scalar
Ensure: $o[m+1]$: LWE ciphertext, where each element is scalar (initialized to zero)

```

1: Local variable:  $oic[nl_k]$  (initialized to zero)
2:  $o[m] = c[n]$  ▷ // Decomposition
3:  $oic = \text{Decompose}(c[0..n], l_k)$ 
4: for  $i = 0$  to  $m+1$  do
5:   for  $j = 0$  to  $n * l_k$  do
6:      $o[i] = o[i] - oic[j] * ksk[j][i]$  ▷ // Vector matrix multiplication

```

the new LWE ciphertext by extracting a specific coefficient from the GLWE

Keyswitching. After performing PBS, the resulting LWE ciphertext is encrypted under a different key, transforming it into a vector of $k \cdot (N+1)$ scalar elements. To decrypt this using the original key, a keyswitching algorithm is required to revert the ciphertext to its original size, which is a vector of $(n+1)$ scalar elements. Algorithm 2 provides a summary of the keyswitching algorithm [10, 29]. The keyswitching process starts by decomposing the input LWE ciphertext. It takes the first $k \cdot N$ elements from the LWE and expands them into a $k \cdot N \cdot l_k$ -element vector, where l_k denotes the decomposition level for keyswitching. This vector is subsequently multiplied by a $k \cdot N \cdot l_k \times (n+1)$ matrix, yielding a vector of $(n+1)$ elements. This vector represents the new LWE ciphertext encrypted with the original key.

2.5 Alternative FHE Schemes

One of the popular FHE schemes is the CKKS algorithm [6], which is known for its ability to support complex and real numbers. Many recent works have addressed the computation challenges in CKKS, including various microarchitectural explorations and accelerators [1, 32, 33, 47, 50]. While these works provide significant performance improvements compared to naive implementations on CPUs or GPUs, the performance gap between FHE and native workloads remains substantial, particularly due to significant bootstrapping overhead. In comparison, this work targets TFHE, which has fundamentally different requirements and trade-offs from CKKS. A high-level qualitative comparison between TFHE and CKKS is summarized in Table 2. From a computational standpoint, TFHE is more lightweight than CKKS, with considerably smaller ciphertext and bootstrapping key sizes, making it more practical for low to mid-range computing systems. Both CKKS and TFHE have computational bottlenecks in polynomial transformations for polynomial multiplication. However, TFHE offers more flexibility as it allows the use of FFT or NTT for polynomial transformation, which opens up more algorithm optimization possibilities.

Another significant difference is in bootstrapping. Bootstrapping represents a significant fraction of the overall execution time for both CKKS and TFHE. Although bootstrapping in TFHE is considerably faster than CKKS, TFHE only allows for bootstrapping one message at a time. On the other hand, CKKS allows for bootstrapping a vector of messages at a time due to ciphertext packing capability, resulting in higher throughput performance. However,

Table 2: Comparison between CKKS and TFHE

	CKKS	TFHE
Ciphertext size	MB level	KB level
Bootstrapping key size	GB level	10s-100s MB level
Computational bottleneck	NTT	FFT/NTT
Ciphertext packing	supported	not supported
Bootstrapping latency	high latency	low latency
Bootstrapping throughput	high throughput	low throughput
Supported operations	add, multiply	add, look-up table
Message type	real, complex	real, integer, boolean

TFHE offers more diverse operations due to PBS, which can evaluate any function of single variable. This unique feature of TFHE schemes allows them to evaluate logical and relational operations, which are difficult to perform in the CKKS scheme.

From application perspective, TFHE is particularly useful for evaluating the activation function in neural networks [12, 36, 49], evaluating tree-based machine learning models [20], or emulating a CPU, which can run encrypted programs [38]. Overall, while CKKS is well-suited for homomorphic encryption where large numbers of the real/complex number are involved, TFHE's flexibility in its message type and support for PBS makes it a practical choice for general-purpose homomorphic encryption applications.

3 MOTIVATION: IDENTIFYING BOTTLENECK IN TFHE

The main bottleneck in TFHE is caused by PBS operations which do not support ciphertext packing, so each message needs to be bootstrapped one by one at a time, eventually resulting in low throughput PBS. Thus, it is clear that the primary goal of TFHE accelerator is to increase the PBS throughput. To accomplish this, we first conduct a workload breakdown analysis of TFHE performance on a CPU to determine the overall workload distribution. Next, we perform an initial study of TFHE performance on a GPU to identify the main bottlenecks when accelerating TFHE on commercial hardware accelerators. We use the Concrete library [11] for TFHE workload breakdown. For the GPU implementation of TFHE, we use the NuFHE library [40], and utilize Nvidia's Nsight profiling tools to obtain kernel execution performance metrics on an Nvidia Titan RTX GPU.

Figure 1 showcases the breakdown of TFHE gate operation and the distribution of execution time on a CPU. At the topmost level of the call stack, around 65% of the computation time is dedicated to PBS, 30% to keyswitching (KS), with the residual 5% allocated to other operations, including homomorphic addition and subtraction. The distribution of execution time might exhibit minor variations based on the selected security parameter. For our experiment, we adopted the default 110-bit security parameter from the NuFHE library as presented in the Concrete library. In our experiment, we utilize the default 110-bit security parameter from the NuFHE library in the concrete library. Within the PBS function, approximately 96% of the execution time is dedicated to blind rotation (BR), while the remaining time is allocated to modulus switching and sample extraction. The blind rotation consists of sequential iterations, where the majority of each iteration's execution time is consumed by the external product. This external product can

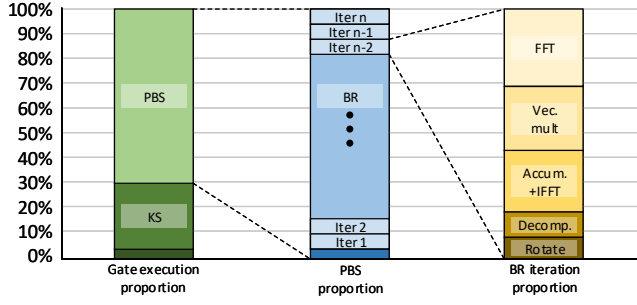


Figure 1: Workload breakdown for TFHE gate operation on CPU, showing the percentage of computation time across various levels of call stack.

be broken down into three parts: FFT, vector multiplication, and IFFT. An imbalance in workload is observed between FFT and IFFT, attributable to the decomposition operation. This operation amplifies the number of polynomials in the test vector from $(k + 1)$ to $(k + 1) \cdot l_b$. The problem stems from the fact that the FFT can only be carried out after this decomposition, leading to an increased number of polynomials requiring FFT processing. At the end of the external product, the output size is reduced to $(k + 1)$ polynomials, which then need to be transformed back using IFFT. The ratio of the number of polynomials processed by FFT and IFFT is $l_b : 1$, making the workload imbalance more severe as l_b increases.

The most time-consuming operation of blind rotation is the external product which performs matrix-vector multiplication with polynomials as elements. This process is executed for every ciphertext. When multiple ciphertexts are considered, one can envision multiple nodes, each performing blind rotation for its respective ciphertext. Since the bootstrapping key (a matrix of polynomials) can be shared among nodes (provided they all work on the same blind rotation iteration), it is possible to batch multiple ciphertexts and perform matrix-matrix multiplication with polynomials in each iteration. This is precisely what we observe in NuFHE implementations when the optimal security parameter was chosen. All GPU cores (SM) work on the same blind rotation iteration, allowing them to share a portion of the bootstrapping key. We refer to this strategy as **device-level batching**: each core works on a different ciphertext's PBS but all cores share the same bootstrapping key in the same iteration of blind rotation.

Figure 2 displays the profiling results for the blind rotation kernel in the NuFHE library. Due to the device-level batching, the execution time is kept constant for the first 72 ciphertexts, as our GPU has 72 cores. Nevertheless, when the number of ciphertexts surpasses the available cores (i.e., all cores are fully utilized), blind rotation (BR) fragmentation occurs, resulting in a twofold increase in the overall execution time for the blind-rotation kernel. The TFHE execution time on a GPU can be expressed as:

$$\text{Total time} = (\#BR \text{ fragmentations}) \times \text{BR time per core} \quad (1)$$

where the number of BR fragmentations is determined by:

$$\#BR \text{ fragmentations} = \left\lceil \frac{\#ciphertexts}{\text{batch size}} \right\rceil \quad (2)$$

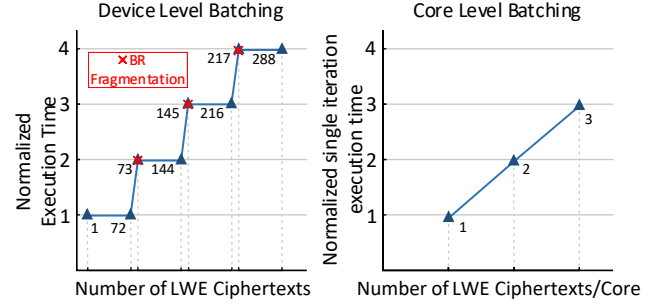


Figure 2: Execution time of the blind rotation kernel on a GPU, illustrating the fragmentation caused by device-level batching and the corresponding increase in execution time when core-level batching is implemented on the GPU.

To boost overall throughput, we can either reduce the number of BR fragmentations or decrease the BR execution time per core. Reducing the BR execution time can be achieved by exploiting parallelization within each blind rotation iteration. On the other hand, reducing the fragmentation number can be accomplished by batching more ciphertexts in a single BR fragment. In the case of NuFHE, the number of ciphertexts that can be batched is always fixed to the number of cores. Increasing the number of cores is effective, but it is costly and not scalable. This consideration leads us to the concept of **core-level batching**, wherein multiple ciphertexts can be batched into a single core, effectively increasing the batch size of a single BR execution to the product of core-level batch size and device-level batch size.

Utilizing core-level batching means allocating multiple ciphertexts to each core, requiring the sequential processing of multiple ciphertexts in a single-blind rotation iteration. In the case of GPUs, even if we reduce the number of BR fragmentation with core-level batching, the total execution time does not decrease at all as the execution time for each iteration in blind rotation increases accordingly, as illustrated in Figure 2. This motivates us to design a specialized TFHE core that can efficiently process a series of ciphertexts. With customized and fully pipelined arithmetic datapath, we can accelerate the sequential ciphertext processing in a core and amortize the hardware cost. Overall, multiple cores process many ciphertexts in parallel, while each core accelerates sequential ciphertext processing with specialized logic.

Besides PBS, another major time-consuming operation in TFHE is the keyswitching operation. Similar to PBS, keyswitching in TFHE is performed individually for each ciphertext. Our measurements indicate that the execution time for keyswitching accounts for approximately 30% of the total execution time. In the case of NuFHE, keyswitching is accelerated using different kernels from those for blind rotation, introducing additional latency. By designing a specialized hardware unit, we can hide the latency arising from the keyswitching operation, which is commonly executed after PBS. Therefore, developing a dedicated hardware accelerator tailored to address these challenges will significantly enhance the efficiency and performance of TFHE operations.

4 STRIX ARCHITECTURE

4.1 Parallelism Strategy

As discussed in Section 3, the blind rotation process involves multiple iterations that cannot be parallelized. In each iteration, matrix-vector multiplications of polynomials must be performed for processing one LWE ciphertext. However, if we consider processing multiple LWE ciphertexts in a single iteration, i.e., input batching, the operation would become a polynomial matrix-matrix multiplication, which is the major workload for the accelerator. Within this notation, we discover four parallelism levels that can be leveraged in the Strix architecture.

Four parallelism levels in PBS. Figure 3 illustrates the four ways to exploit parallelism in matrix-matrix multiplication of polynomials. The most straightforward parallelism to leverage is test-vector level parallelism (*TvLP*), which is the number of rows that can be processed concurrently in the test-vector matrix. For each row in the test-vector matrix, it is essential to consider the number of polynomials that can be processed in parallel, referred to as polynomial level parallelism (*PLP*). Moreover, for each polynomial, the number of coefficients that can be processed in parallel, known as coefficient level parallelism (*CLP*), must be considered. Finally, the number of columns in the output matrix that can be processed in parallel, called column level parallelism (*CoLP*), should also be taken into account.

Hardware implication on parallelism levels. Although all four levels of parallelism can be utilized simultaneously, each level has different implications for hardware implementation and cost. Exploiting excessive parallelism at a certain level can lead to expensive hardware cost. For example, *TvLP* is same as the number of processing cores in the architecture, *CLP* is associated with the number of lanes the computing unit, and *PLP* and *CoLP* are related to the replication the computing unit as will be discussed in Section 5.

Parallelism availability. The degree of parallelism that can be achieved for *PLP* and *CoLP* is determined by the size of the vector or matrix utilized in the external product calculation. The maximum number of polynomials that can be processed in parallel for *PLP* is $(k + 1) \times l_b$, while the maximum number for *CoLP* is $(k + 1)$. Based on the latest TFHE library [11], the typical range of k is between 1 and 4, while the typical range of l_b is between 2 and 4. Therefore, we can expect a minimum parallelism of *PLP* = 4 and *CoLP* = 2. The availability of parallelism for the *CLP* can be achieved when performing FFT on an N degree polynomial. The practical range for N lies between 1,024 and 16,384. In the case of *TvLP*, parallel processing can only be utilized when executing the PBS operation on multiple ciphertexts. The degree of parallelism for *TvLP* is dependent on the number of inputs, which varies by application. For instance, in a machine learning context, the image size of the MNIST dataset is 28×28 , allowing for parallel execution of 784 PBS, resulting in a maximum *TvLP* value of 784.

Prioritization of the parallelisms. In order to create an efficient hardware accelerator, it is important to set the design goal and focus on it (e.g., whether it targets reducing latency or increasing throughput). We believe that prioritizing throughput is more important in the TFHE accelerator, as many applications require multiple ciphertexts PBS. To this end, we prioritize parallelism in the order

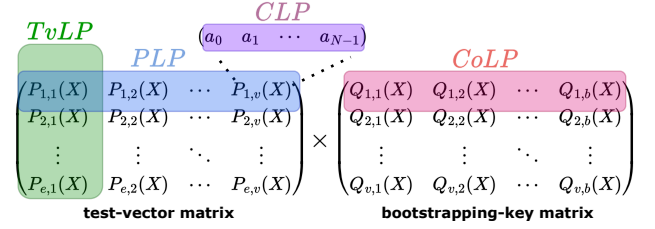


Figure 3: Illustration of four levels of parallelism when performing external product.

of *TvLP*-*CLP*-*PLP*-*CoLP*, taking into account parallelism availability, hardware cost, and throughput goals. *TvLP* and *CLP* offer more significant parallelism opportunities than *PLP* and *CoLP*, based on common security parameters and applications. When choosing between *TvLP* and *CLP*, we consider bandwidth and implications for the hardware. Increasing *TvLP* involves adding more cores to process more LWE ciphertexts in one epoch and increase throughput. Increasing *CLP* also improves throughput performance, however it also results in a need for more external bandwidth due to the shorter duration between blind rotation iterations, which requires the bootstrapping key to be retrieved more frequently. Finally, considering the typical value of TFHE parameters and implementation feasibility, such as area, bandwidth, and interconnect cost, we chose *TvLP* = 8, *CLP* = 4, *PLP* = 2, and *CoLP* = 2.

Parallelism in Keyswitching. When dealing with multiple ciphertexts, keyswitching becomes a matrix-matrix multiplication. Our strategy for parallelism in keyswitching is similar to that of external products, except without polynomial level parallelism (*PLP* = 1) as we are no longer working with polynomials. For keyswitching processing, we use *TvLP* = 8, *CLP* = 8, and *CoLP* = 8 in our Strix architecture. The *TvLP* value is determined by the number of cores, with the goal of having one keyswitch's functional unit cluster on each core. Meanwhile, *CLP* is set to match the throughput of functional units from PBS computation. Finally, *CoLP* is set as large as possible to utilize all channels from HBM fully.

4.2 Strix Architecture

Identifying four parallelisms and workload characteristics of many ciphertexts' PBS, we propose Strix architecture optimized for stream processing of LWE. Figure 4 depicts Strix's overall architecture. It includes multiple Homomorphic Streaming Cores (HSC), a global scratchpad for storing keys and ciphertexts, network-on-chip (NoC) for distributing the keys to HSCs, and HBM as an external memory.

Specialized functional units for TFHE. It is essential to have specialized functional units for optimizing computation and data reuse during PBS and keyswitching operations. We analyzed the workload breakdown of PBS and keyswitching to identify their shared similarities. Based on our observations, we determined that at least five distinct functional units are required to process both PBS and keyswitching. These functional units are ① Rotator Unit, ② Decomposer Unit, ③ I/FFT Unit (I/FFTU), ④ Vector Multiply-Add (VMA) Unit, and ⑤ Accumulator Unit.

As the name suggests, the Rotator Unit is used to perform polynomial negacyclic rotation and subtraction during blind rotation computation. The Decomposer Unit, employed in both PBS and

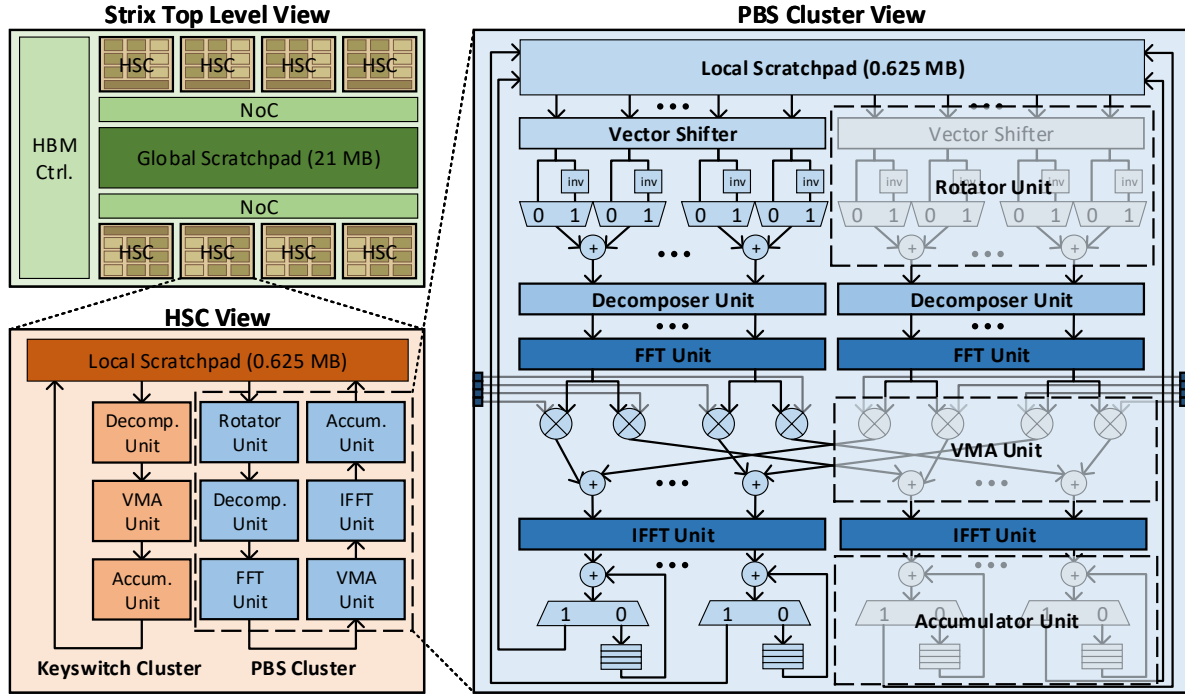


Figure 4: Overall architecture of Strix presented in three perspectives: Top level view, Homomorphic Streaming Core (HSC) view, and PBS cluster view. Each functional unit within the HSC is fully pipelined and duplicated based on either CoLP or PLP parallelism to achieve optimal throughput.

keyswitching computations, decomposes input polynomials into several smaller value polynomials. The I/FFT Unit is utilized exclusively in blind rotation computation to transform polynomials into the frequency domain for accelerating polynomial multiplication based on the convolution theorem. The Vector Multiply-Add Unit is used in both keyswitching and PBS computations to perform element-wise multiplication between two vectors (or polynomials) and accumulate their partial sums. Finally, the Accumulator Unit aggregates all partial sum values to obtain the final sum.

Homomorphic Streaming Core (HSC). All specialized functional units are instantiated within the HSC, which is responsible for executing both PBS and keyswitching operations. As depicted in Figure 4, an HSC consists of a local scratchpad for storing test-vectors and ciphertexts, along with two compute clusters: the keyswitch cluster and the PBS cluster. The PBS cluster is equipped with six-stage fully-pipelined functional units, designed to process a stream of ciphertexts in a dataflow manner without any stalling to advance the throughput of core-level batching. A full traversal of the entire pipeline in the PBS cluster corresponds to one iteration of blind rotation. We partition the computation process of a blind rotation iteration into six stages to achieve optimal workload balancing.

As discussed in Section 3, there is a workload imbalance issue between FFT and IFFT, because of the decomposition prior to FFT. To address this, we split the polynomial accumulation between the frequency and time domains rather than accumulating everything in the frequency domain. This approach can yield a 1:1 workload ratio between the FFT and IFFT. Additionally, we implement a

separate keyswitch cluster using a pipeline of three functional units similar to those in the PBS cluster. By incorporating separate compute clusters for both keyswitching and PBS, we can conceal the keyswitching execution time behind the PBS's and enhance the HSC's overall throughput.

Memory system and NoC. Strix has a two-level memory structure: global scratchpad and local scratchpad. The global scratchpad is composed of several shared and private memory sections. The shared memory section of the global scratchpad stores the bootstrapping key (bsk) and keyswitching key (ksk), which are shared among all cores. To distribute the ksk and bsk among the cores, we use a fixed multicast network, as the communication is one-to-all and unidirectional. The private memory section of the global scratchpad stores the LWE ciphertexts and the initial test-vector inputs for each core. Since the private memory section is exclusive to each core, we utilize a point-to-point network for reading and writing data between the private memory sections and the cores.

Conversely, the local scratchpad, located within each core, stores intermediate test-vectors and LWE ciphertexts. Similar to the global scratchpad, the local scratchpad contains separate sections allocated for the PBS cluster and keyswitch cluster memory. The PBS cluster memory is used exclusively for storing intermediate test-vectors generated during blind rotation computations. In the final iteration of blind rotation, the output LWE ciphertext is stored directly in the keyswitch cluster memory, initiating the keyswitching operation. As depicted in Figure 4, the local scratchpad can only be accessed by specific functional units, and since the communication pattern

between functional units and the local scratchpad is fixed, a point-to-point network can be employed.

4.3 Workload Scheduling

In any privacy-preserving computation using TFHE, the workload can be regarded as a series of sequential PBS and keyswitching operations. Taking the neural network example, the network is first transformed into a series of linear and nonlinear operations. Linear operations, such as homomorphic addition and scalar multiplication on ciphertexts, can be executed rapidly compared to nonlinear operations. Nonlinear operations, which involve evaluating the activation function, are necessary for both PBS and keyswitching.

Strix schedules the workload in a series of epochs, with each epoch containing a maximum number of LWE ciphertexts equal to the product of device-level and core-level batch sizes. The device-level batch size is determined by the number of compute cores (HSCs), while the core-level batch size depends on the number of test-vectors that can be stored in the local scratchpad, which varies based on the chosen TFHE parameters. The PBS cluster within each core executes n iterations of blind rotation, processing a core-level batch size of LWE ciphertexts in a pipeline through all functional units in the PBS cluster during each iteration.

Once the blind rotation is complete, the output is forwarded to the keyswitching cluster to begin the keyswitching process, while simultaneously initiating a new blind rotation for the next epoch to hide the latency of the keyswitching process. The keyswitching operation is typically faster than the blind rotation, as it involves standard matrix-matrix multiplication without any polynomial operations. The keyswitch cluster computes the matrix-matrix multiplication by tiling it into a series of smaller matrix-matrix multiplications and working on each smaller matrix as a sequence of vector-matrix multiplications. During these processes, both the bootstrapping key and keyswitching key are distributed from the global scratchpad, while the next portions of the bootstrapping and keyswitching keys for the subsequent iteration and tile are fetched from external memory. After all operations are complete, the output is collected in the global scratchpad and written back to the host machine.

5 MICROARCHITECTURE

5.1 I/FFT Unit

Strix utilizes the FFT algorithm to compute the polynomial multiplication. It is well-known that the FFT algorithm consists of many similar computations called butterfly, which operates on two complex-number inputs and the twiddle factor of FFT. Even though this offers the opportunity for parallelism, the required input and the connection between the butterfly are not straightforward. In general, the FFT algorithm heavily relies on data shuffling to match up the connection between the butterfly. As the number of points of FFT increases, it is difficult to do all the computations simultaneously. It is required to store the partial results and reload them when they are needed again for the butterfly operation. However, it causes an irregular memory access pattern due to the characteristics of the butterfly connection in FFT. To solve this problem, we design the pipelined-FFT, which has the computation through all the stages in a pipeline manner. With this architecture, the data

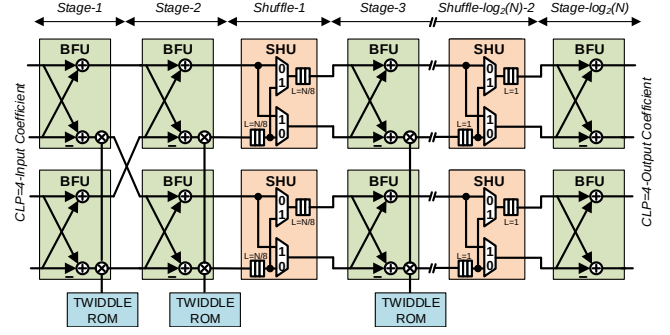


Figure 5: N-point of pipelined-(I)FFT unit

shuffle can be done inside the (I)FFT unit because the partial result is directly needed for the next stage butterfly.

Figure 5 shows the FFT/IFFT unit based on the pipelined-FFT architecture [21]. This unit exploits the four-lane input coefficient of polynomial corresponding to the $CLP = 4$ parallelism. It consists of $\log_2(N)$ stage of butterfly units (BFUs), which associate with all the stages computation in N -point FFT. The number of butterflies in each stage depends on the number of CLP . In this case ($CLP = 4$), we need 2 BFUs per stage. All the data-path in the unit consist of two fixed-point numbers for the real and imaginary parts. The shuffle unit (SHU) connects the different stages of the butterfly. It shuffles the data depending on the number of delays, i.e., L , in each stage. It has two L -delay elements and two multiplexers to alternate the data stream from the upper input and bottom input. The multiplexer selector signal is commuted every L clock cycles to shuffle the input stream. In the implementation, we use a shift register for the small delay element ($L < 32$) and use an SRAM-based shift register for the large delay element ($L \geq 32$) for the efficiency of the area and energy. We have the Twiddle-ROM at every stage to accommodate that every BFU needs different twiddle factor. Although this architecture requires additional memory space, it achieves high throughput. The initial latency for transforming an N degree polynomial is $\frac{N}{CLP}$. Afterward, it can transform an N degree polynomial every $\frac{N}{CLP}$ clock cycles consecutively, maintaining maximum throughput.

Folding optimization. In general, every polynomial can be separated into four components: real-even, imaginary-even, real-odd, and imaginary-odd polynomial. FFT has symmetry properties that can map these components cleanly to the frequency domain[27, 42]. When performing polynomial transformations, these symmetry properties can be used to combine two polynomials in a single FFT operation, placing one polynomial in the real part and another in the imaginary part. This optimization method is used in the latest TFHE library[11]. Although this mapping appears efficient because it conserves hardware resources by transforming two polynomials with a single FFT unit, an additional hardware module is needed after the Fourier transform to separate the first and second polynomials. This extra module requires memory buffers to reorder the polynomials, significantly increasing overall latency and area.

A more effective way to utilize the imaginary slot in the Fourier transform is to employ the folding scheme[34]. This scheme splits an N degree polynomial into two $\frac{N}{2}$ degree polynomials, placing one polynomial as the imaginary component of the other, effectively

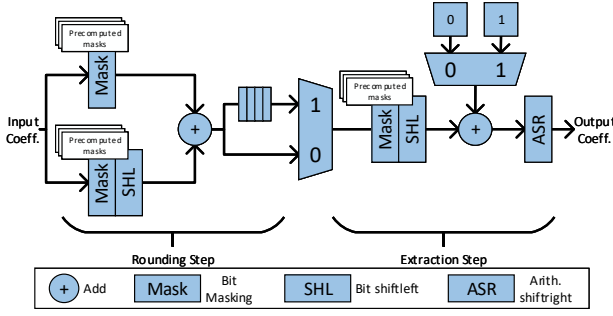


Figure 6: Microarchitectural details of a single lane within the fully pipelined decomposer unit.

folding and reducing the polynomial length by half. Although this means we need as many FFT modules as there are polynomials to transform in parallel, it eliminates the additional hardware overhead for polynomial separation. Moreover, by halving the polynomial length, we can implement smaller FFT modules in the hardware.

The folding scheme allows our FFT units to virtually support $CLP = 8$ by only implementing $CLP = 4$, while also reducing the size of the FFT unit. In our case, the FFT unit was initially designed to support up to 2^{14} -point polynomial transformations, but we reduced it to 2^{13} -point transformations by applying the folding scheme. Although we must implement $CLP = 8$ for other functional units besides the FFT unit to fully benefit from the folding scheme, increasing CLP for other functional units is less expensive compared to increasing CLP in the FFT unit. Consequently, the folding scheme substantially reduces the size of our FFT unit by almost 50%.

5.2 Decomposer Unit

The decomposer unit receives a polynomial as input and decomposes it into l_b polynomials. However, this decomposition differs from the commonly used RNS decomposition in CKKS accelerators. The decomposition process for each coefficient in the polynomial is carried out according to the following expression:

$$\left\| a - \sum_{i=1}^{l_b} \left\lfloor \frac{a}{Q/B^i} \right\rfloor_B \right\|_{\infty} \leq \frac{Q}{B^{l_b}} \quad (3)$$

where a is the polynomial's coefficient, Q is the original ciphertext modulo, B is the new basis for decomposition, and l_b is the decomposition level.

The challenge lies in designing a fully pipelined decomposer unit that can accept input and produce output every cycle, thereby maintaining maximum throughput. Additionally, the decomposition process must be executed for each coefficient in the polynomial. As evident, the required division and rounding operations are quite complex and expensive. To address this issue, we propose a novel decomposer unit that divides the process into two steps: rounding and extraction, as illustrated in Figure 6. In order to mitigate the high costs, we implement the entire circuit without using any multipliers. By employing a combination of masking, shifting, and adding, we reduce both the circuit cost and overall complexity.

In the rounding step, the coefficient term is rounded to the nearest representable value according to the chosen TFHE parameter. We use a combination of masks to extract the required upper bits and carry to obtain the rounded coefficient. This rounded coefficient is then decomposed into l_b coefficients across multiple clock cycles, and we store them in a buffer for later use. In the extraction step, we utilize a series of prepared masks to extract specific bit portions. After obtaining the extracted bit, we add it to the carry (zero or one) from the previous extracted bit to derive the final decomposed value. The decomposer unit operates for $\frac{N}{CLP} \times l_b$ cycles for each polynomial. To align the throughput with the rotator and FFT units, we implement the decomposer unit with $2 \times CLP$ lanes, achieving a chip-wide throughput of $2 \times CLP \times CoLP \times ToLP$ data per clock cycle.

5.3 Rotator, VMA, and Accumulator Unit

Rotator unit. The rotator unit, located within the PBS cluster, is responsible for carrying out negacyclic rotation and polynomial subtraction as in Algorithm 1. As depicted in Figure 4, the rotator unit's datapath employs eight lanes ($2 \times CLP$) for processing eight coefficients in parallel, aligning with the folding scheme's requirements. In each clock cycle, the rotator unit reads two coefficients from each memory bank for both polynomials, performs cyclic rotation to align the coefficients with the lanes, and adds/subtracts polynomials. With $CoLP = 2$, we utilize two instances of the rotator unit, achieving a chip-wide throughput of $2 \times CLP \times CoLP \times ToLP$ coefficients per clock cycle and one initiation interval.

VMA unit. The VMA unit is designed to compute the vector-matrix multiplication of polynomials between test-vector and bsk polynomials. Coefficients of the bsk polynomial are sourced from the global scratchpad and distributed via the multi-cast network. The VMA units in the PBS cluster and keyswitching cluster have different data types, with the former utilizing complex multipliers and adders and the latter employing integer multipliers and adders. Figure 4 illustrates the VMA unit's architecture, which includes multipliers followed by an adder tree that accumulates the results into a partial sum. With $PLP = 2$, two instances of the VMA unit are utilized, yielding a chip-wide throughput of $CLP \times PLP \times ToLP$ complex coefficients per clock cycle.

Accumulator unit. Once the VMA unit has accumulated data via an adder tree, the output is directed to the IFFT unit for transformation back into the time domain as a polynomial. The IFFT unit's output is then sent to the accumulator unit for final accumulation in the time domain. The accumulator unit consists of $2 \times CLP$ input lanes, each containing an accumulator with a buffer (see Figure 4). Each buffer can store $\frac{N}{2 \times CLP}$ coefficients. When the final sums are obtained, they are forwarded back to the local scratchpad for use as input in the next blind rotation iteration. By utilizing two instances of the accumulator unit with $CoLP = 2$, the overall throughput rate is $2 \times CLP \times CoLP \times ToLP$ coefficients per clock cycle.

6 EVALUATION

6.1 Hardware Modeling

We implement Strix's functional units, scratchpads, and interconnections in System Verilog and synthesize them using the TSMC 28nm process design kit. All functional unit datapaths are 32-bit,

Table 3: The area and power breakdown of Strix.

Component	Area (mm ²)	Power (W)
Local scratchpad (0.625MB)	0.92	0.47
Rotator	0.02	0.01
Decomposer	0.28	0.02
I/FFTU	7.23	5.49
VMA	0.63	0.10
Accumulator	0.32	0.13
1 HSC	9.38	6.21
8 cores	75.03	49.67
Global NoC	0.04	0.01
Global scratchpad (21MB)	51.40	26.24
HBM2 PHY	14.90	1.23
Total	141.37	77.14

Table 4: TFHE parameter set for the experiments.

	I	II	III	IV
n	500	630	592	991
k	1	1	1	1
N	2^{10}	2^{10}	2^{11}	2^{14}
l_b	2	3	3	2
λ	80-bit	110-bit	128-bit	128-bit

except for the FFTU which uses 64-bit and twiddle 16-bit to ensure accuracy. We have then further optimized our design to minimize the area and power consumption while maintaining 1.2 GHz clock frequency. We construct the global scratchpad with a total capacity of 21 MB as a dual-port (one read, one write), multi-bank 128-bit width SRAM. It is also implemented as double-buffered to store bsk, ksk, and (R)LWE ciphertext during computation to decouple off-chip memory access and the computation. Assuming a modest bandwidth of 300 GB/s for one HBM2e stack[2], we utilize 8 channels for bsk's transfer, 4 channels for ksk's transfer, and the remaining 4 channels for transferring ciphertexts. Moreover, we implement a scratchpad inside each HSC as a true dual-port multi-bank 32-bit SRAM with 0.625 MB capacity. We implement two separate data buses to broadcast bsk and ksk, where each bus has 512-bit and 256-bit widths, respectively. Table 3 shows the synthesis result of the Strix with 8 HSCs. Overall, it takes 141.37 mm² area and consumes up to 77.14 W power.

6.2 Experimental Methodology

To test the performance of Strix, we create a custom cycle-level simulator and model the accelerator in an object-oriented manner to determine the latency, bandwidth, and throughput of each functional unit. Specifically, the simulator converts the input workload as a computational graph with nodes, where each node mainly represents either bootstrapping or keyswitching or a combination of both operations. Each node in the graph will further be decomposed into several epochs.

Table 5: Comparison of PBS latency and throughput on a variety of platforms.

	Platform	Parameter Set	Latency (ms)	Throughput (PBS/s)
Concrete[11]	CPU	I	14.00	70
		II	19.00	52
		III	38.00	26
		IV	969.00	1
NuFHE[40]	GPU	I	37.00	2,000
		II	700.00	500
YKP[51]	FPGA	I	1.88	2,657
		III	4.78	836
XHEC[39]	FPGA	I	–	2,200
		II	–	1,800
MATCHA[28]	ASIC	I	0.20	10,000
Strix	ASIC	I	0.16	74,696
		II	0.23	39,600
		III	0.44	21,104
		IV	3.31	2,368

To evaluate the performance of Strix, we conduct microbenchmark and application benchmark tests. The microbenchmark assesses the latency and throughput of PBS and keyswitching operations. The simulator allows us to set the desired number of LWE ciphertexts and various TFHE parameters to compute the required number of epochs. We compared the results of Strix with previous works, including the original Concrete library (Intel Xeon Platinum)[10], nuFHE (Nvidia Titan RTX)[40], YKP (FPGA)[51], XHEC (FPGA)[39], and MATCHA (ASIC)[28].

Table 4 specifies the TFHE parameter sets we used for the experiments. Parameter set I serves as a baseline comparison, which has been used in all prior works. Set II and III are specific case parameters used by YKP and XHEC, respectively. Meanwhile, set IV is new TFHE parameters tested on Strix, with the highest polynomial degree corresponding to a heavier computational workload and better precision. We also perform experiments to evaluate the effects of FFT optimization on our design, using parameter set I. Additionally, we showcase our timing measurements for functional units to demonstrate their utilization and bandwidth efficiency. For the application benchmark, we conducted tests on Zama Deep-NN models with 20, 50, and 100-layer depths, using the same TFHE parameters and polynomial degrees ($N = 2^{10}$, $N = 2^{11}$, and $N = 2^{12}$) as in the reference paper [12].

6.3 Results and Discussions

Throughput and latency. Table 5 displays the latency and throughput performance of Strix in comparison to previous works. As observed, Strix achieves superior performance in terms of throughput and latency compared to all prior works. Additionally, Strix supports a wider range of TFHE parameter sets than any other previous accelerator. When compared to the previous state of the art, MATCHA, which only supports parameter set I, Strix attains a 7.4× improvement in throughput and a 1.25× reduction in latency. The modest latency enhancement is expected since Strix only utilizes a modest amount of $CLP = 4$. Conversely, the GPU implementation, NuFHE, solely supports TFHE parameters with a

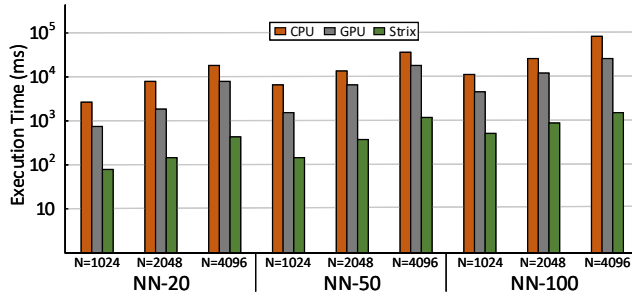


Figure 7: Application benchmark results using Zama Deep-NN models [12]. The neural network execution time was compared against CPU and GPU.

Table 6: FFT optimization effects on performance.

Metric	No Folding	With Folding	Improvement
Latency (ms)	0.27	0.16	1.68×
Throughput (PBS/s)	37,472	74,696	1.99×
FFT Unit Area (mm ²)	3.13	1.81	1.73×
Total Core Area (mm ²)	13.87	9.38	1.48×

fixed polynomial degree $N = 2^{10}$ and is optimized only for $l_b = 2$. Changing the parameter set from I to II in NuFHE significantly reduces performance, as they discard the blind rotation kernel and execute the entire blind rotation using the FFT kernel sequentially. Lastly, Strix exhibits its exceptional performance for the largest TFHE parameter set IV, improving the throughput and latency by 2,368× and 292×, respectively, in comparison to Concrete.

Zama Deep-NN. Zama Deep-NN has three different models: NN-20, NN-50, and NN-100, which indicate the depth of the neural network. The input consists of 28×28 pixels, where each pixel is encrypted with one cipher. The first layer performs 10×11 convolution followed by ReLU activation, producing an output image of dimensions $[1, 2, 21, 20]$. The remaining layers are dense layers with 92 neurons on each layer, followed by ReLU activation between each layer. To perform every ReLU operation, we utilize PBS. We conduct three different TFHE parameter setups for each model, increasing the polynomial size and the security level, which increases the workload. The parameters for the experiment are detailed in [12]. Figure 7 shows the execution time results of running Zama Deep-NN on Strix. In all cases, Strix outperforms the CPU and GPU implementations. We observe that Strix speeds up the evaluation time for NN by 33–38× compared to CPU and 8–17× compared to GPU.

FFT optimization impact on area and performance. Unlike leveled HE schemes that utilize NTT, TFHE can employ FFT for polynomial multiplication, presenting new opportunities for algorithm-hardware optimization. We implement a folding scheme in our FFT algorithm, as explained in Section 5. To assess the impact of the folding scheme on overall performance and design, we create two types of Strix: non-folded Strix and folded Strix, both with $CLP = 4$ for the 2^{14} -point FFT unit. Table 6 compares non-folded Strix and folded Strix in terms of latency, throughput, FFT unit area, and total core area, which have been improved by 1.68×, 1.99×,

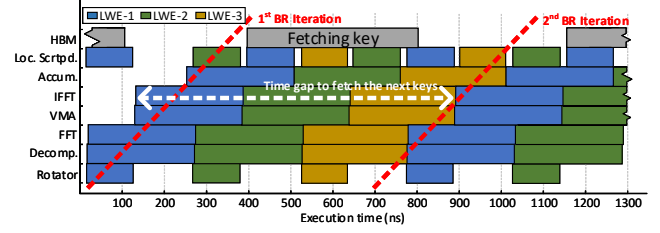


Figure 8: The experiment measures the timing of all functional units for the first two iterations of blind rotation, assuming that each core processes three LWE ciphertexts in each iteration.

Table 7: The TvLP and CLP effects on the required external bandwidth, latency, and throughput.

TvLP	CLP	Required Bandwidth (GB/s)	Latency (ms)	Throughput (PBS/s)
16	2	200	7.2	2,368
8	4	257	3.8	2,368
4	8	371	3.8	2,364
2	16	599	3.6	1,240
1	32	1,053	3.6	620

1.73×, and 1.48×, respectively. Without folding optimization, all units in the core must utilize a 4-lane implementation to match the bandwidth of the 4-lane FFT unit, affecting the final latency and throughput performance. Additionally, a 2^{14} -point FFT unit would cost twice more hardware resources than our current design.

Overall timing and bandwidth utilization. We conducted measurements to analyze how Strix’s hardware and bandwidth are being utilized. Figure 8 shows the overall timing for the first two iterations of blind rotation within each core, assuming that each core processed three LWE ciphertexts (represented by different colors) using parameter set I. All functional units in Strix are highly utilized, with the decomposer, I/FFT, VMA, and accumulator units achieving close-to 100% utilization rates, while the rotator unit reached a 50% utilization rate. The local scratchpad was also in high demand, being accessed about 90% of the time by the rotator unit for read accesses and the accumulator unit for write accesses. Moreover, the HBM bandwidth was occupied around 60% of the time, primarily used for transferring the bootstrapping key during blind rotation. For each iteration, the bootstrapping key needed to be fetched only once and was then reused for all LWE ciphertexts. The HBM bandwidth utilization was significantly influenced by the number of LWE ciphertexts processed for each iteration and the number of lanes (CLP). If the number of LWE ciphertexts is small or CLP is high, the system would be bottlenecked by HBM (i.e., memory-bound) as the time gap to fetch the next key became smaller. Conversely, if the number of LWE ciphertexts is large or CLP is low, the system would be bottlenecked by the (I)FFT units (i.e., compute-bound).

TvLP vs CLP trade off. We instantiate several versions of Strix with a variety of $TvLP$ and CLP values to examine how they affect throughput and latency performance when the maximum external

bandwidth is fixed to 300 GB/s, and to determine the required bandwidth for each particular $TvLP$ and CLP configuration. We perform this experiment using parameter set IV. Table 7 illustrates how a variety of $TvLP$ and CLP values impact performance while keeping the product of $TvLP$ and CLP constant. When $TvLP = 16$ and $CLP = 2$, Strix becomes heavily compute-bound, requiring only 200 GB/s bandwidth and yielding a latency performance of 7.2 ms. We observe that the $TvLP = 8$ and $CLP = 4$ configuration represents the sweet spot, balancing the compute and memory requirements. It requires a bandwidth of 257 GB/s, achieving 3.8ms latency while sustaining high throughput of 2,368 PBS/s. The remaining $TvLP$ and CLP configurations with CLP values greater than 4 render the system memory-bound, demanding bandwidth higher than 300 GB/s and causing underutilization of the compute core. In the extreme case of instantiating only one core ($TvLP = 1$ and $CLP = 32$), the required bandwidth is about 1 TB/s, making the workload heavily memory-bound and resulting in a throughput drop to only 620 PBS/s. As in our Strix implementation, the configuration of $TvLP = 8$ and $CLP = 4$ provides the best performance given the 300 GB/s external memory bandwidth.

7 RELATED WORKS

CKKS-based accelerator. Recent research on FHE accelerators primarily focuses on accelerating CKKS schemes with designs like FAB[1], CraterLake[47], and ARK[32]. Although direct comparison is difficult, CKKS accelerators likely perform better on large machine learning workloads due to inherent packing operations. However, TFHE supports more applications with its universal operations and can be designed with lower hardware costs, particularly in memory footprint. Our design requires 26 MB of on-chip memory, compared to FAB, CraterLake, and ARK's 45 MB, 256 MB, and 512 MB, respectively. TFHE's working set can be less than 1 MB, allowing further memory reduction under resource constraints. CKKS acceleration demands higher external memory bandwidth as it is memory-bound[16], while TFHE is compute-bound. Most CKKS accelerators need two stacks of HBM2e for 1 TB/s bandwidth, while our design requires only one stack of HBM2e 300 GB/s.

TFHE-based accelerator Various works have attempted to accelerate the TFHE scheme on platforms like GPU [11, 15, 40], FPGA [22, 39, 51], and ASIC [28]. NuFHE's GPU acceleration uses either NTT or FFT, with FFT outperforming NTT [40]. In FPGA acceleration, [51] employs NTT for polynomial multiplication, achieving a 1.3× throughput improvement compared to NuFHE. ASIC acceleration in MATCHA [28] outperforms prior accelerators using bootstrapping key unrolling [4] that reduces blind rotation iterations at the cost of increased key size. This paper presents Strix, which advances TFHE acceleration by utilizing two-level batching and specialized functional units. Our streaming architecture-based accelerator supports various TFHE parameters and achieves 7.4× higher throughput than MATCHA.

8 CONCLUSION

Strix is the first accelerator specifically designed to enable high-throughput PBS for the TFHE scheme. We first identify the challenges of accelerating TFHE on GPUs and recognize the blind rotation fragmentation problem due to limited batching capabilities. To

address this issue, we propose device-level batching and core-level batching to significantly increase the batch size. However, to efficiently implement core-level batching, specialized and optimized functional units are required. To achieve this, we analyze the parallelism levels in PBS and identify four levels of parallelism that can be exploited. Based on these insights, we design specialized, fully-pipelined functional units that enable streaming ciphertext processing for high-throughput performance. Strix has achieved 1,067× and 37× throughput improvements compared to CPUs and GPUs, respectively, and attained 38× and 17× latency improvements when evaluating deep neural networks compared to CPUs and GPUs, respectively.

ACKNOWLEDGMENTS

The authors would like to express their sincere gratitude to Gilbert Jonatan for the guidance in using profiling tools, to Junsoo Kim and Sukbin Lim for setting up the design compiler environment, and to the anonymous reviewers for the constructive comments. This work was supported in part by the Information Technology Research Center (ITRC) support program (IITP-2020-0-01847), in part by the Ministry of Science and ICT (MSIT), South Korea, under the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant (No. 2022-0-01037), and in part by Samsung Advanced Institute of Technology, Samsung Electronics Co., Ltd..

REFERENCES

- [1] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2022. FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption. *arXiv preprint arXiv:2207.11872* (2022).
- [2] JEDEC SOLID STATE TECHNOLOGY ASSOCIATION. 2013. *JEDEC Standard: HBM DRAM*. Technical Report. JEDEC.
- [3] Daniel J Bernstein. 2009. Introduction to post-quantum cryptography. In *Post-quantum cryptography*. Springer, 1–14.
- [4] Florian Bourse, Michele Minelli, Matthias Minihold, and Pascal Paillier. 2018. Fast homomorphic evaluation of deep discretized neural networks. In *Annual International Cryptology Conference*. Springer, 483–512.
- [5] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* 6, 3 (2014), 1–36.
- [6] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*. Springer, 409–437.
- [7] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. 2016. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *international conference on the theory and application of cryptology and information security*. Springer, 3–33.
- [8] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. 2016. A homomorphic LWE based E-voting scheme. In *Post-Quantum Cryptography*. Springer, 245–265.
- [9] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. 2017. Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 377–408.
- [10] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. 2020. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology* 33, 1 (2020), 34–91.
- [11] Ilaria Chillotti, Marc Joye, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2020. CONCRETE: Concrete operates on ciphertexts rapidly by extending TfhE. In *WAHC 2020–8th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, Vol. 15.
- [12] Ilaria Chillotti, Marc Joye, and Pascal Paillier. 2021. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *International Symposium on Cyber Security Cryptography and Machine Learning*. Springer, 1–19.

- [13] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2021. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for tfhe. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 670–699.
- [14] Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel Paul Smart, and Barry Van Leeuwen. 2022. Scooby: Improved Multi-Party Homomorphic Secret Sharing Based on FHE. *Cryptology ePrint Archive* (2022).
- [15] Wei Dai and Berk Sunar. 2015. cuHE: A homomorphic encryption accelerator library. In *International Conference on Cryptography and Information Security in the Balkans*. Springer, 169–186.
- [16] Leo de Castro, Rashmi Agrawal, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, Chirag Juvekar, and Ajay Joshi. 2021. Does Fully Homomorphic Encryption Need Compute Acceleration? *arXiv preprint arXiv:2112.06396* (2021).
- [17] Léo Ducas and Daniele Micciancio. 2015. FHEW: bootstrapping homomorphic encryption in less than a second. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 617–640.
- [18] Alessandro Falchetta and Manuel Roveri. 2022. Privacy-preserving deep learning with homomorphic encryption: An introduction. *IEEE Computational Intelligence Magazine* 17, 3 (2022), 14–25.
- [19] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive* (2012).
- [20] Jordan Frery, Andrei Stoian, Roman Bredehoft, Luis Montero, Celia Kherfallah, Benoît Chevallier-Mames, and Arthur Meyre. 2023. Privacy-Preserving Tree-Based Inference with Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2023/258. <https://eprint.iacr.org/2023/258> <https://eprint.iacr.org/2023/258>.
- [21] Mario Garrido. 2021. A survey on pipelined FFT hardware architectures. *Journal of Signal Processing Systems* (2021), 1–20.
- [22] Serhan Gener, Parker Newton, Daniel Tan, Silas Richelson, Guy Lemieux, and Philip Brisk. 2021. An FPGA-based Programmable Vector Engine for Fast Fully Homomorphic Encryption over the Torus. In *SPSL: Secure and Private Systems for Machine Learning (ISCA Workshop)*.
- [23] Craig Gentry. 2009. A fully homomorphic encryption scheme. Stanford university.
- [24] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.
- [25] Craig Gentry and Shai Halevi. 2011. Implementing gentry's fully-homomorphic encryption scheme. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 129–148.
- [26] Craig Gentry, Amit Sahai, and Brent Waters. 2013. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Annual Cryptology Conference*. Springer, 75–92.
- [27] Haitao Guo, Gary A Sittin, and C Sidney Burrus. 1998. The quick Fourier transform: an FFT based on symmetries. *IEEE transactions on signal processing* 46, 2 (1998), 335–341.
- [28] Lei Jiang, Qian Lou, and Nrushad Joshi. 2022. MATCHA: A Fast and Energy-Efficient Accelerator for Fully Homomorphic Encryption over the Torus. *arXiv preprint arXiv:2202.08814* (2022).
- [29] Marc Joye. 2022. SoK: Fully homomorphic encryption over the [discretized] torus. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2022), 661–692.
- [30] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2021), 114–148.
- [31] Subhash Khot. 2005. Hardness of approximating the shortest vector problem in lattices. *Journal of the ACM (JACM)* 52, 5 (2005), 789–808.
- [32] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. *arXiv preprint arXiv:2205.00922* (2022).
- [33] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An accelerator for bootstrappable fully homomorphic encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 711–725.
- [34] Jakub Klemsa. 2021. Fast and Error-Free Negacyclic Integer Convolution Using Extended Fourier Transform. In *International Symposium on Cyber Security Cryptography and Machine Learning*. Springer, 282–300.
- [35] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* 10 (2022), 30039–30054.
- [36] Qian Lou and Lei Jiang. 2019. She: A fast and accurate privacy-preserving deep neural network via leveled tfhe and logarithmic data representation. *arXiv:1906.00148* (2019).
- [37] Chiara Marcolla, Victor Sucasas, Marc Manzano, Riccardo Bassoli, Frank HP Fitzek, and Najwa Aaraj. 2022. Survey on Fully Homomorphic Encryption, Theory, and Applications. *Proc. IEEE* (2022).
- [38] Kotaro Matsuoka, Ryotaro Banno, Naoki Matsumoto, Takashi Sato, and Song Bian. 2021. Virtual Secure Platform: A {Five-Stage} Pipeline Processor over {TFHE}. In *30th USENIX Security Symposium (USENIX Security 21)*. 4007–4024.
- [39] Kevin Nam, Hyunyoung Oh, Hyungon Moon, and Yunheung Paek. 2022. Accelerating N-Bit Operations over TFHE on Commodity CPU-FPGA. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (San Diego, California) (ICCAD '22)*. Association for Computing Machinery, New York, NY, USA, Article 98, 9 pages. <https://doi.org/10.1145/3508352.3549413>
- [40] nucypher. 2020. NuFHE, a GPU-powered Torus FHE implementation. <https://github.com/nucypher/nufhe>
- [41] Robert Podschwadt, Daniel Takabi, Peizhao Hu, Mohammad H Rafiei, and Zhipeng Cai. 2022. A Survey of Deep Learning Architectures for Privacy-Preserving Machine Learning with Fully Homomorphic Encryption. *IEEE Access* (2022).
- [42] L Rabiner. 1979. On the use of symmetry in FFT computation. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 27, 3 (1979), 233–239.
- [43] Oded Regev. 2009. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)* 56, 6 (2009), 1–40.
- [44] M Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An architecture for computing on encrypted data. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1295–1309.
- [45] Ronald L Rivest, Len Adleman, Michael L Dertouzos, et al. 1978. On data banks and privacy homomorphisms. *Foundations of secure computation* 4, 11 (1978), 169–180.
- [46] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A fast and programmable accelerator for fully homomorphic encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 238–252.
- [47] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. 2022. CraterLake: a hardware accelerator for efficient unbounded computation on encrypted data. In *ISCA*. 173–187.
- [48] Kaustubh Shivdikar, Gilbert Jonatan, Evelio Mora, Neal Livesay, Rashmi Agrawal, Ajay Joshi, José L Abellán, John Kim, and David Kaeli. 2022. Accelerating polynomial multiplication for homomorphic encryption on gpus. In *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 61–72.
- [49] Andrei Stoian, Jordan Frery, Roman Bredehoft, Luis Montero, Celia Kherfallah, and Benoît Chevallier-Mames. 2023. Deep Neural Networks for Encrypted Inference with TFHE. *Cryptology ePrint Archive*, Paper 2023/257. <https://eprint.iacr.org/2023/257> <https://eprint.iacr.org/2023/257>.
- [50] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical Homomorphic Encryption Accelerator. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 870–881.
- [51] Tian Ye, Rajgopal Kannan, and Viktor K. Prasanna. 2022. FPGA Acceleration of Fully Homomorphic Encryption over the Torus. In *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–7. <https://doi.org/10.1109/HPEC55821.2022.9926381>